

Метод К-средних (K-means)

Алгоритм кластеризации данных

Работу выполнили студентки группы 5030102/30202:
Ятманова Анна, Киселева Ксения

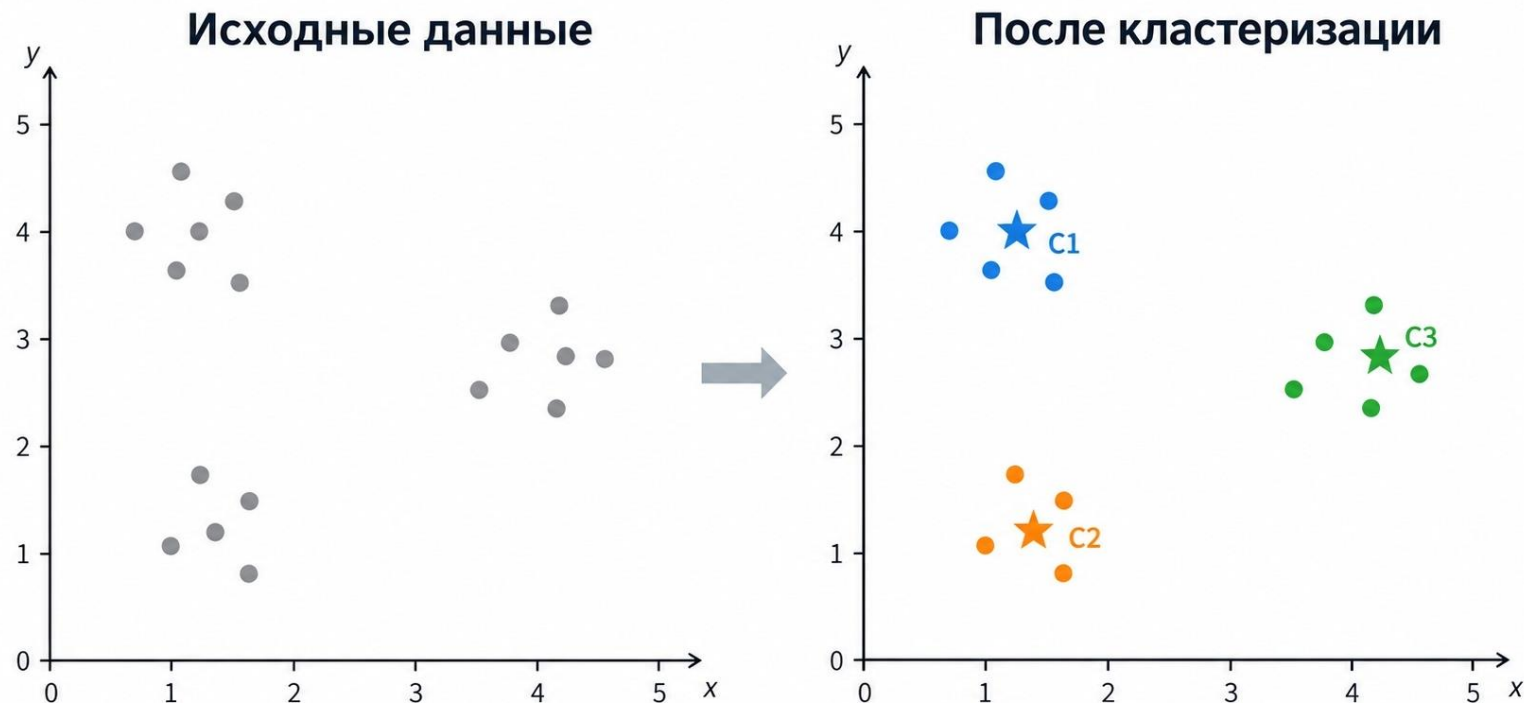
Кластеризация

Дано: $X = \{x_1, x_2, \dots, x_n\}$

Необходимо: разделить
объекты на K групп

$C_1, C_2, \dots, C_k$ так, чтобы:

- Объекты внутри одного кластера были похожи;
- Объекты разных кластеров отличались.



Применения K-means



● сегментация клиентов;



● группировка документов;



● анализ поведения пользователей;



● анализ изображений;



● поиск структуры в данных;



● предварительный анализ перед построением других моделей.

Основная идея K-means

Каждый кластер представлен своим центром – центроидом. • • • → ★

Центроид – средняя точка кластера:

$\mu_j = \frac{1}{|C_j|} \sum x_i$, где C_j - j-й кластер (набор точек), $|C_j|$ - число точек в кластере, x_i - точка из кластера.

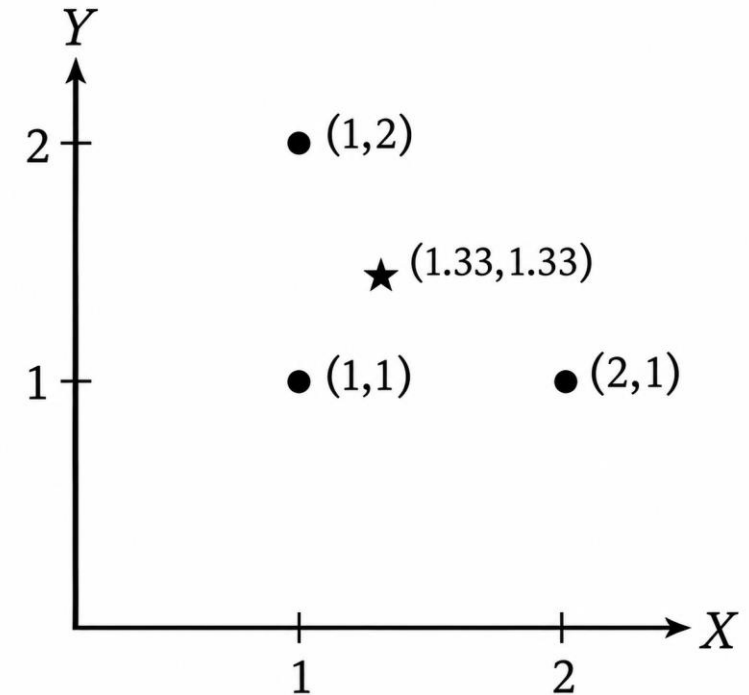
Пример:

Точки: (1, 1), (1, 2), (2, 1)

$$\mu = \left(\frac{1+1+2}{3}, \frac{1+2+1}{3} \right) = (1.33, 1.33)$$

Правило отнесения точки: точка относится к тому кластеру, чей центроид ближе.

Расстояние: $d(x, \mu) = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2 + \dots}$



Что минимизирует K-means?

Целевая функция: $J = \sum_{j=1}^K \sum_{x_i \in C_j} ||x_i - \mu_i||^2$

Названия этой величины:

- WCSS (Within-Cluster Sum of Squares)
- SSE (Sum of Squared Errors)
- inertia

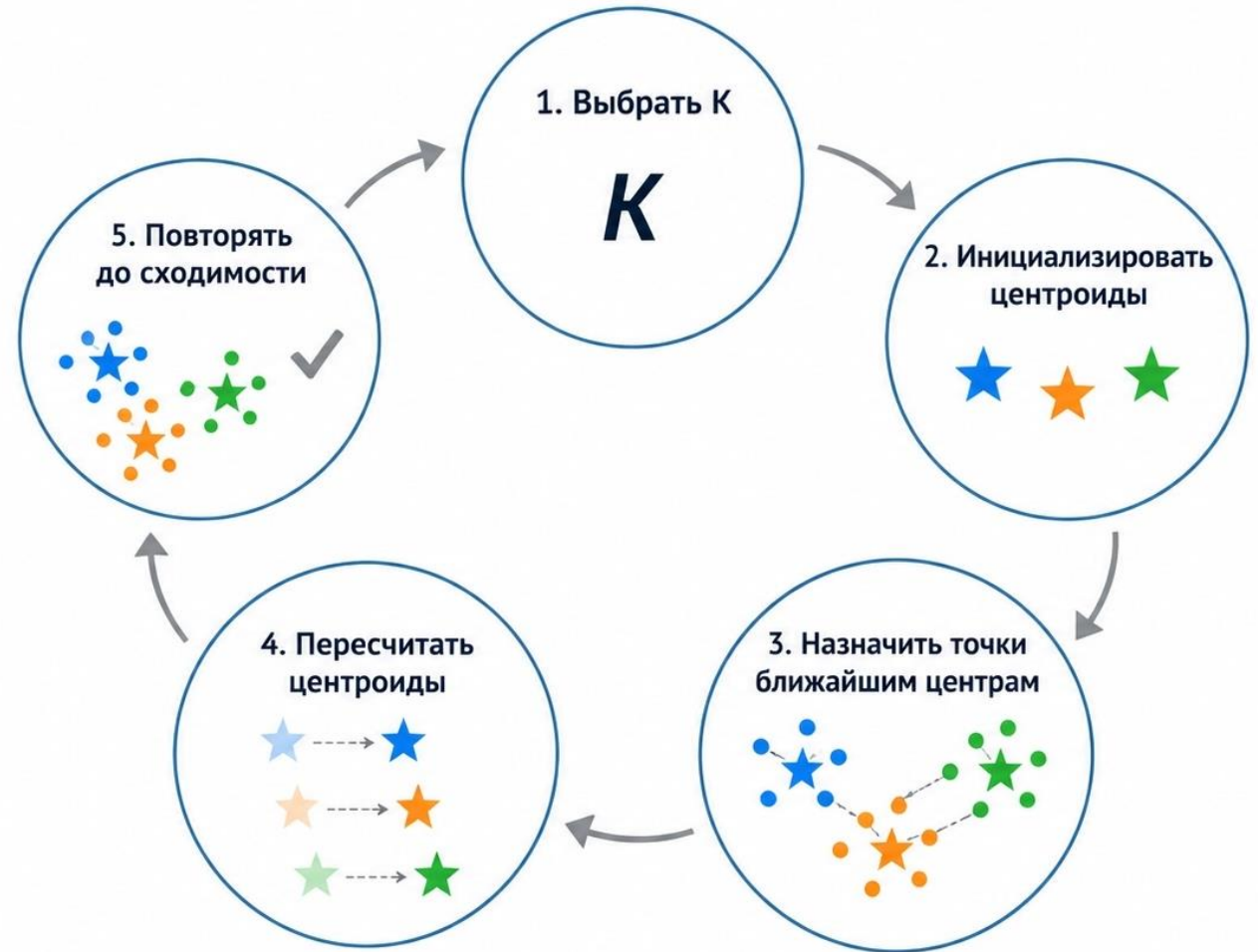
Смысл: хотим сделать объекты максимально близкими к центрам своих кластеров.

Чем меньше J — тем компактнее кластеры.

Алгоритм K-means

1. Выбрать количество кластеров K
2. Выбрать начальные центроиды
3. Assignment step — назначить каждую точку ближайшему центроиду
4. Update step — пересчитать центры кластеров
5. Повторять шаги 3–4 до сходимости

Алгоритм K-means



Численный пример: исходные данные

Берем $K=2$

Точка	x	y
A	1	1
B	1	2
C	2	1
D	5	5
E	6	5
F	5	6

Начальные центры:

$$\mu_1 = (1, 1)$$

$$\mu_2 = (2, 1)$$



Итерация 1: распределение точек

Расстояние: $d(x, \mu) = \sqrt{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2}$

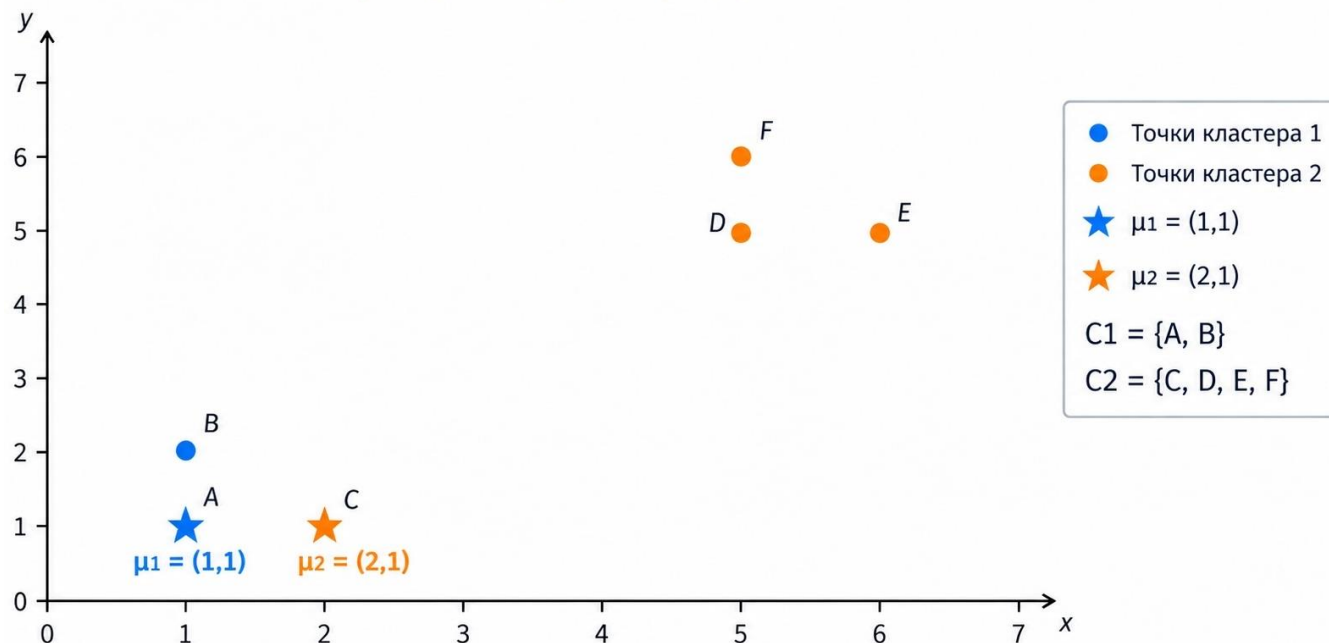
Точка	До μ_1	До μ_2	Кластер
A	0	1.00	1
B	1.00	1.41	1
C	1.00	0	2
D	5.66	5.00	2
E	6.40	5.66	2
F	6.40	5.83	2

Получили:

$$C_1 = \{A, B\}$$

$$C_2 = \{C, D, E, F\}$$

Итерация 1: распределение точек



Пример расчёта для точки B:

- до μ_1 : $\sqrt{(1 - 1)^2 + (2 - 1)^2} = 1$
 - до μ_2 : $\sqrt{(1 - 2)^2 + (2 - 1)^2} = \sqrt{2} \approx 1.41$
- > B идёт в кластер 1

Итерация 1: пересчет центроидов

Второй кластер $C_2 = \{C, D, E, F\}$:

$$\mu_2 = \left(\frac{2 + 5 + 6 + 5}{4}, \frac{1 + 5 + 5 + 6}{4} \right) = (4.5, 4.25)$$

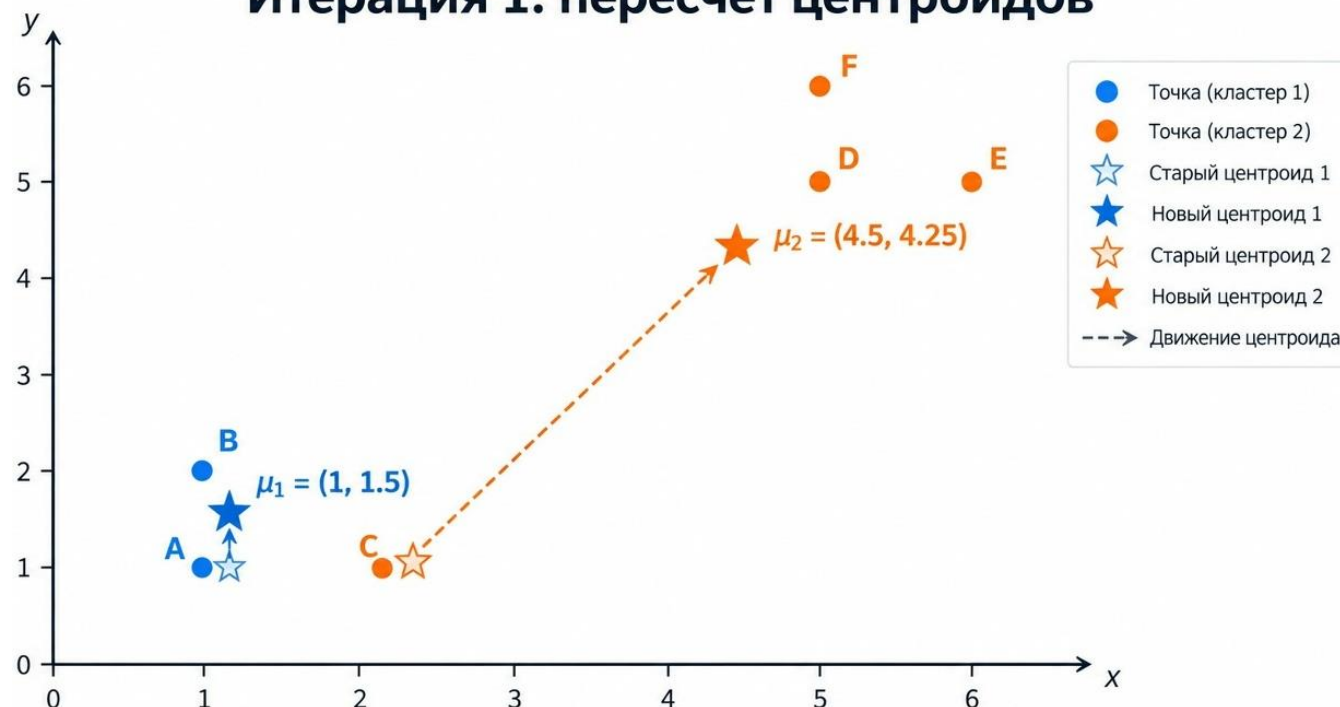
Первый кластер $C_1 = \{A, B\}$:

$$\mu_1 = \left(\frac{1 + 1}{2}, \frac{1 + 2}{2} \right) = (1, 1.5)$$

Новые центры:

$$\begin{aligned}\mu_1 &= (1, 1.5) \\ \mu_2 &= (4.5, 4.25)\end{aligned}$$

Итерация 1: пересчет центроидов



Итерация 2

Новые центры:

$$\mu_1 = (1, 1.5); \mu_2 = (4.5, 4.25)$$

Поверяем точку С:

$$d(C, \mu_1) = 1.12; d(C, \mu_2) = 4.10$$

-> С переходит из кластера 2 в кластер 1.

Новое распределение:

$$C_1 = \{A, B, C\}$$

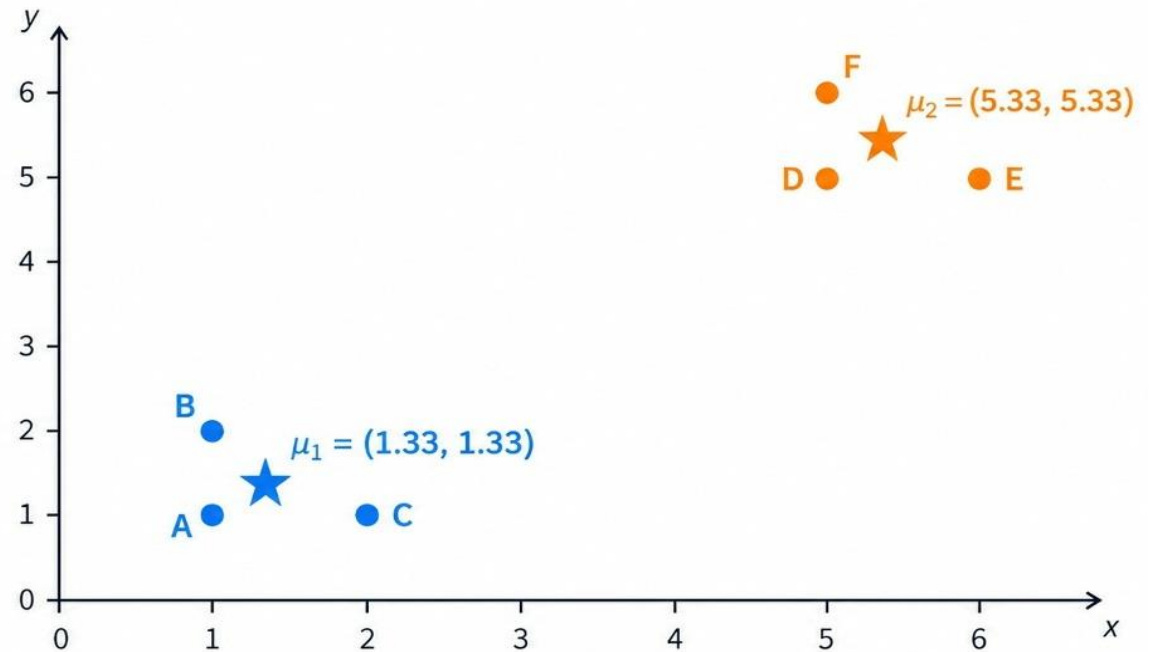
$$C_2 = \{D, E, F\}$$

Перерасчет центров:

$$\mu_1 = \left(\frac{1 + 1 + 2}{3}, \frac{1 + 2 + 1}{3} \right) = (1.33, 1.33)$$

$$\mu_2 = \left(\frac{5 + 6 + 5}{3}, \frac{5 + 5 + 6}{3} \right) = (5.33, 5.33)$$

Итерация 2: итоговое разбиение



Алгоритм сошелся

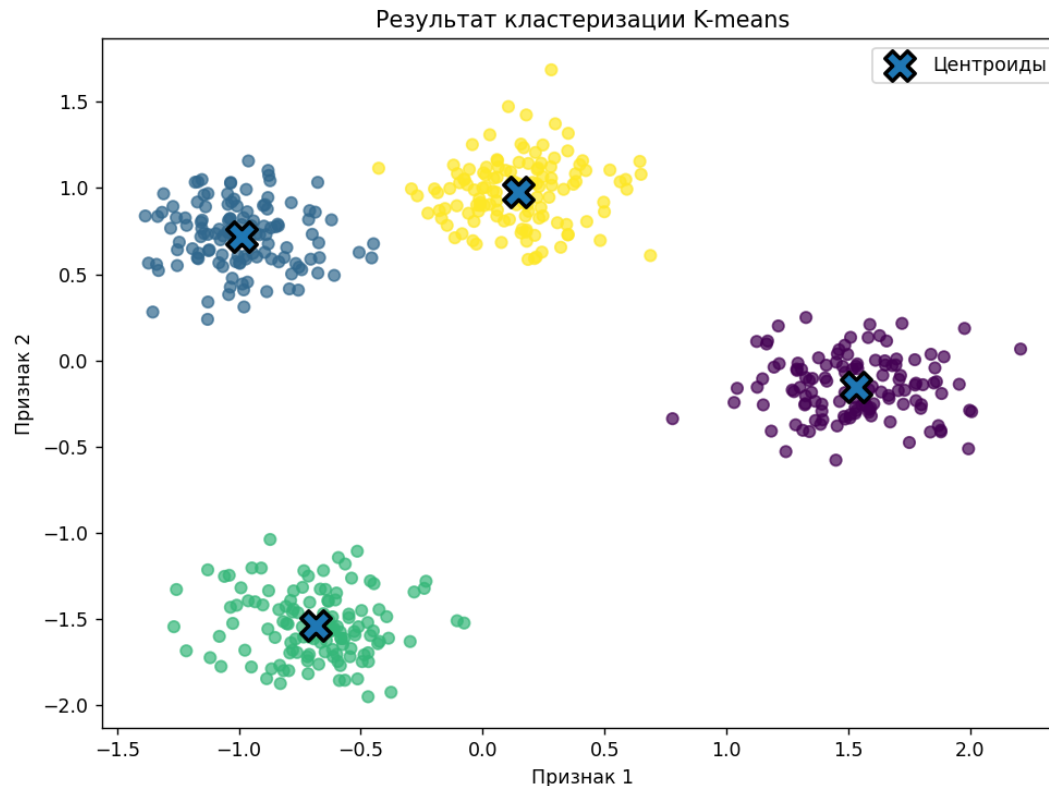
Значения целевой функции J	
Этап	J
Начальное распределение	92
После пересчёта центров	24.25
После окончательного распределения	≈ 2.67
Вывод: на каждой итерации решение становится лучше – J уменьшается	

Реализация на Python

```
from sklearn.cluster import KMeans
```

```
model = KMeans(  
    n_clusters=4,  
    init="k-means++",  
    n_init=10,  
    random_state=42  
)  
labels = model.fit_predict(X)  
print(model.cluster_centers_)  
print(model.inertia_)
```

- `n_clusters` — количество кластеров;
- `fit_predict()` — обучает модель и возвращает номера кластеров;
- `cluster_centers_` — координаты центроидов;
- `inertia_` — значение целевой функции.

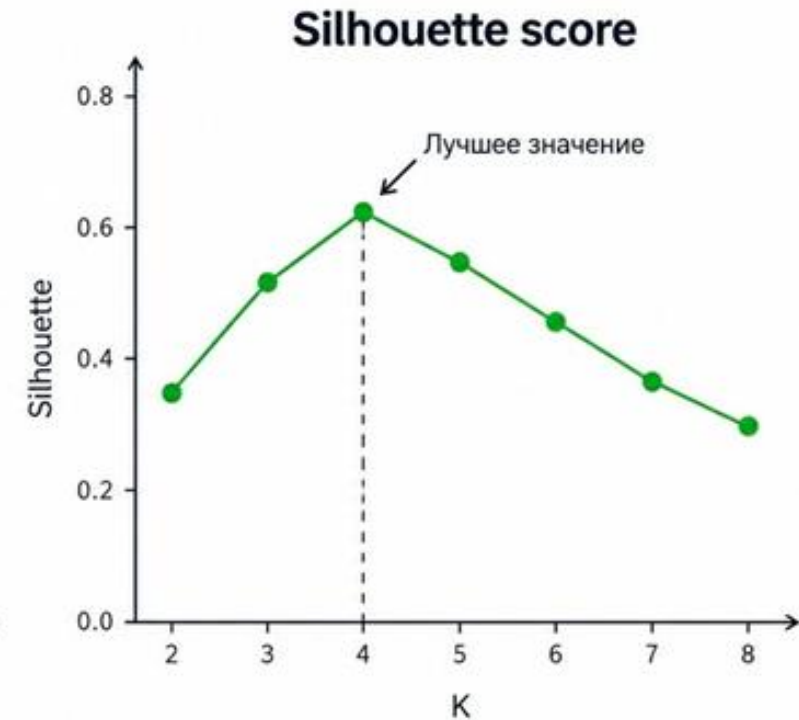
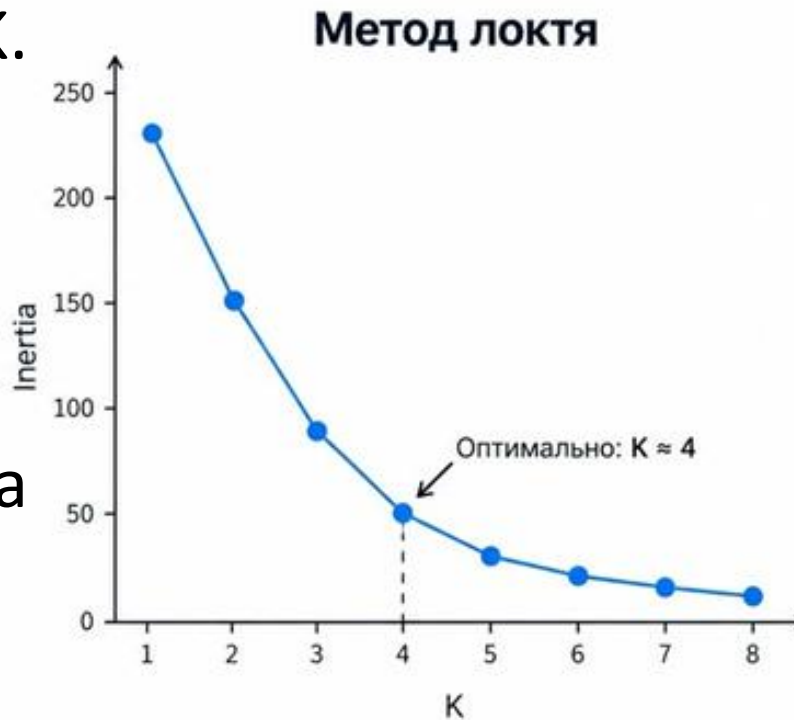


Как выбрать K?

Проблема: K-means требует заранее указать количество кластеров.

- Inertia всегда уменьшается с ростом K.
- Ищем момент, после которого улучшение становится незначительным.
- Универсального способа выбора K нет — часто решает смысл задачи.

Как выбрать K?



Silhouette score: чем ближе к 1 — тем лучше разделены кластеры ($-1 \leq s \leq 1$)

Масштабирование данных

Клиент	Возраст	Доход
A	18	30 000
B	25	70 000
C	40	150 000
D	55	220 000
E	70	300 000

K-means основан на расстояниях → масштаб признаков влияет на результат

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

После StandardScaler ни один признак не доминирует только из-за единиц измерения.

Клиент	Возраст	Доход
A	-1.24	-1.26
B	-0.87	-0.86
C	-0.08	-0.04
D	0.70	0.67
E	1.49	1.49

Недостатки K-means

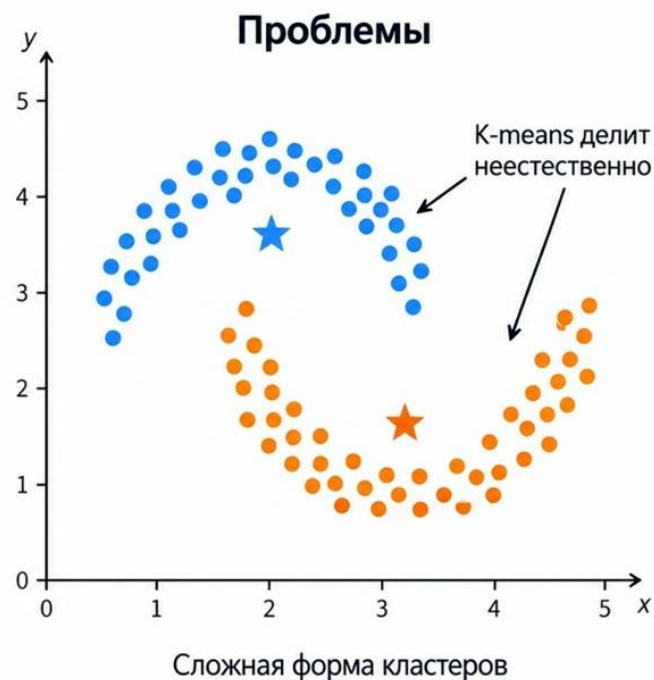
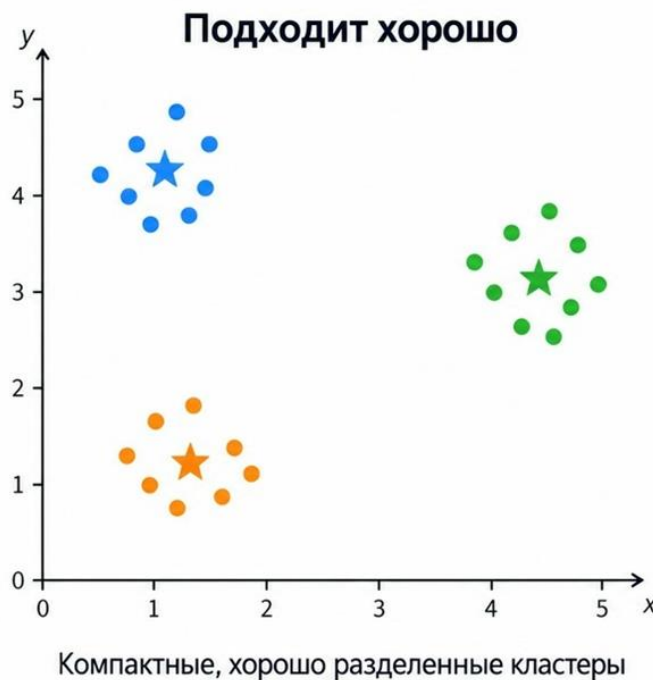
Работает хорошо, если кластеры

- ✓ Компактные
- ✓ Примерно сферические
- ✓ Сопоставимого размера

Проблемы

- × Нужно заранее знать K
- × Чувствительность к выбросам
- × Чувствительность к масштабу
- × Зависимость от инициализации
- × Плохо работает с кластерами сложной формы

Когда K-means работает плохо

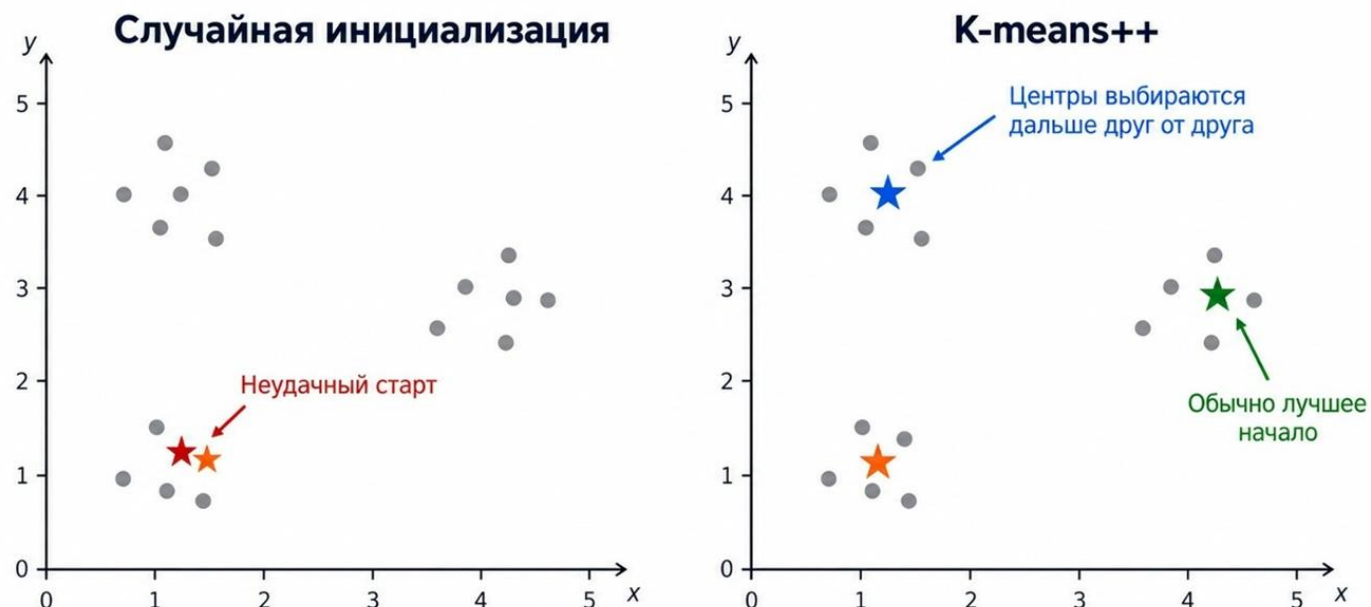


K-means++

Случайная инициализация

- центроиды могут оказаться рядом;
- возможен плохой локальный минимум;
- результат зависит от старта.

Инициализация: случайно vs K-means++



K-means++

- первый центр выбирается случайно;
- следующие центры с большей вероятностью выбираются **далеко от уже выбранных**;
- обычно получаем более удачное начальное положение.

Альтернативы

Метод	К заранее	Форма кластеров	Выбросы	Особенность
K-means	Да	Только выпуклые, примерно круглые/сферические	Очень чувствителен – выброс сдвигает центр	Самый быстрый
DBSCAN	Нет	Любой формы, включая кольца, дуги, цепочки	Помечает выбросы как «шум» и не включает в кластеры	Работает по плотности
Иерархический	Нет (выбирается после)	Зависит от linkage: вытянутые, «цепочкой»	Зависит от linkage	Строит дерево кластеров (дендрограмму)
GMM	Да (число компонент)	Эллиптические	Чувствителен	Даёт вероятность принадлежности к каждому кластеру
K-medoids	Да	Как у K-means	Устойчивее: выброс не тянет центр так сильно	Центр – реальный объект из данных, а не среднее

Что использовать на больших данных?

Проблема обычного K-means:

- на каждой итерации обрабатывается весь набор данных;
- при очень большом числе объектов это долго;
- возрастают требования к памяти и времени.

Решение: MiniBatch K-means

- использует небольшие случайные подвыборки (*mini-batches*);
- обновляет центроиды постепенно;
- сохраняет основную идею K-means;
- работает быстрее на больших данных.

	K-means	MiniBatch K-means
Использует	весь датасет	маленькие батчи
Скорость	ниже	выше
Память	больше	меньше
Точность	выше	немного ниже
Когда применять	средние данные	большие данные

Когда выбирать K-means?

K-means хорошо подходит, когда:

- ✓ Данные числовые
- ✓ Объектов много
- ✓ Нужен быстрый алгоритм
- ✓ Кластеры компактные
- ✓ Есть понимание примерного K

Лучше выбрать другой метод:

- × Много выбросов
- × Сложная геометрия кластеров
- × Неизвестно число групп
- × Нужна вероятностная принадлежность

Выводы

K-means:

1. Разбивает данные на K кластеров
2. Каждый кластер представлен центроидом
3. Поочередно:
 - Назначает объекты
 - Пересчитывает центры
4. Минимизирует внутрикластерную сумму квадратов расстояний
5. Быстрый и простой, но имеет ограничения

K-means – хороший baseline для кластеризации числовых данных.